Python Topic 4 — Booleans, Comparison & Logical Operators

Full Stack Python + AI Roadmap • Taught in 3 layers: New to Python → Intermediate → Expert

🌱 Layer 1 — New to Python

A **boolean** is a yes/no value: `True` or `False` — note the **capital first letter** (unlike JS's lowercase `true` / `false`).

```
is_logged_in = True
is_admin = False
```

You get booleans by **comparing** things:

```
5 > 3        # True
5 < 3        # False
5 == 5       # True   (== means "equal to")
5 != 3       # True   (!= means "not equal to")
```

Combine conditions with plain English words — `and`, `or`, `not`:

```
age = 25
age > 18 and age < 65    # True  (both must be true)
age < 13 or age > 60     # False (at least one must be true)
not is_admin             # True  (flips it)
```

ℹ️ **Big JS difference:** Python uses the words `and`, `or`, `not` instead of `&&`, `||`, `!`.

👩‍💻 Layer 2 — Intermediate (real code + how it works)

Comparison operators (match JS)

```
a == b     # equal
a != b     # not equal
a > b      # greater than
a < b      # less than
a >= b     # greater or equal
a <= b     # less or equal
```

Logical operators — words, not symbols

Python	JavaScript	Meaning
`and`	`&&`	both true
`or`	`||`	at least one true
`not`	`!`	flips the value

```
// JS
if (age > 18 && hasLicense) { ... }

# Python
if age > 18 and has_license:
    ...
```

Truthiness — every value is truthy or falsy

These are the **falsy** values (everything else is truthy):

```
# All FALSY:
False, None, 0, 0.0, "", [], {}, ()
# empty string, empty list, empty dict, zero, None → all falsy
```

So you can check emptiness naturally:

```
name = ""
if not name:
    print("Name is empty!")   # runs — empty string is falsy

items = [1, 2, 3]
if items:
    print("List has stuff!")  # runs — non-empty list is truthy
```

✅ **Cleaner than JS:** instead of `if (arr.length === 0)`, you write `if not items`.

`==` vs `is` (recall from Topic 1)

```
a == b    # do they have the same VALUE?
a is b    # are they the SAME object in memory?
```

Use `==` for value comparison (almost always), and `is` **only** for `None`, `True`, `False`:

```
if x is None:      # ✅ correct way to check for None
if x == None:      # works but not idiomatic — avoid
```

🧠 Layer 3 — Expert (internals & gotchas)

1. `and` / `or` return a VALUE, not just True/False

```
"hello" and "world"    # "world"  → last value if all truthy
"" and "world"         # ""       → first falsy value
"hello" or "backup"    # "hello"  → first truthy value
"" or "backup"         # "backup" → first truthy (skips falsy)
None or "default"      # "default"
```

- `or` returns the **first truthy** value (or the last if all falsy)
- `and` returns the **first falsy** value (or the last if all truthy)

This gives the classic "default value" pattern (like JS `||`):

```
username = input_name or "Guest"   # empty input_name → use "Guest"
```

2. The `or` default trap with `0`

```
count = 0
value = count or 10     # 10 (!) — 0 is falsy, even though 0 is valid!
```

🚨 **Trap:** if `0`, `""`, or `False` are *valid* values, `or` wrongly replaces them. Same bug as JS `count || 10` — but Python has no `??` (nullish coalescing). Use an explicit `is not None` check.

```
value = count if count is not None else 10   # 0 → stays 0 ✅
```

3. Short-circuit evaluation saves you from crashes

```
user = None
if user and user.is_admin:    # safe! `and` stops at `user`, never touches .is_admin
    ...
```

Because `and` stops once it hits a falsy value, `user.is_admin` is never reached when `user` is `None` — no crash. Python's version of optional-chaining logic.

4. `not` precedence and chaining

```
not 5 > 3       # False → reads as: not (5 > 3) = not True = False
1 < 2 < 3       # True  → chained, same as 1<2 and 2<3
```

5. Comparing different types (no coercion!)

```
1 == 1.0        # True  → int and float compare by value
1 == "1"        # False → int vs str, never equal (no coercion!)
```

ℹ️ **Safer than JS:** JS loose `==` makes `1 == "1"` **true**. Python **never** coerces types in `==`, so `1 == "1"` is always `False`. No `"0" == false` madness.

🎯 Interview Q&A Cards

Q: What do `and` / `or` actually return?

A: One of the operands, not a plain bool. `or` returns the first truthy (or last if all falsy); `and` returns the first falsy (or last if all truthy). This enables default-value patterns.

Q: What are Python's falsy values?

A: `False, None, 0, 0.0, "", [], {}, ()` — empty containers, zero, None, and False. Everything else is truthy.

Q: Why can `x or default` be a bug?

A: If `x` is a valid falsy value like `0` or `""`, `or` replaces it with the default. Python has no `??`; use `x if x is not None else default`.

Q: Does Python coerce types in `==`?

A: No. `1 == "1"` is `False` (int vs str). Only numeric types compare cross-type (`1 == 1.0` is True). Much safer than JS loose equality.

🪝 **Memory hook:** *`or` grabs the first truthy, `and` grabs the first falsy. Empty things are falsy. `==` never lies about types.*

✅ **One-liner for interviews:** "Python uses `and` / `or` / `not` keywords. `and` / `or` short-circuit and return one of the operands (not a boolean), enabling default-value patterns. Falsy values are `False, None, 0, 0.0, '', [], {}, ()`. Python never coerces types in `==`, unlike JS."